

OpenDccCLib

Developer Guide — Command Station

From first project to a fully featured DCC command station

Covers all supported platforms and IDEs

Table of Contents

1. Introduction
 - 1.1 What the Library Does (and Does Not Do)
 - 1.2 Platform Support
2. DCC Protocol Concepts
 - 2.1 Signal Encoding
 - 2.2 Packet Format
 - 2.3 Address Types
 - 2.4 Instruction Types
3. Project File Structure
4. dcc_user_config.h in Depth
 - 4.1 Role and Service Mode Flags
 - 4.2 Scheduler Slot Count
 - 4.3 Auto-Refresh Locos
 - 4.4 RailCom Buffer
 - 4.5 Service Mode Parameters
5. Initialization — command_station.c
 - 5.1 The dcc_config_t Struct
 - 5.2 Per-Channel Hardware (dcc_output_hw_t)
 - 5.3 RailCom Hardware (dcc_railcom_hw_t)
 - 5.4 Setup and Main Loop
6. ISR Architecture
 - 6.1 The 58μs Shared Timer
 - 6.2 RailCom One-Shot Timer
 - 6.3 100ms Periodic Tick
7. Implementing the Drivers
 - 7.1 Platform Driver Functions
 - 7.2 Pin Toggle
 - 7.3 Track Power
 - 7.4 Current Sense
 - 7.5 Timestamp
8. The Bit Encoder

- 9. The Scheduler
 - 9.1 Slot Allocation
 - 9.2 Priority System
 - 9.3 Duplicate Combining
 - 9.4 Auto-Refresh
- 10. Service Mode Programming
 - 10.1 Direct Mode
 - 10.2 Paged Mode
 - 10.3 Register Mode
 - 10.4 Address Mode
 - 10.5 ACK Detection
- 11. RailCom (Brief Overview)
- 12. Callbacks — Where Your Application Lives
- 13. Application API Reference
- 14. Porting to a New MCU
- 15. Unit Testing
- 16. Troubleshooting

1. Introduction

OpenDccCLib is a C library that implements the NMRA DCC (Digital Command Control) protocol for microcontrollers. It is designed to be portable across any hardware platform and IDE, with no dynamic memory allocation — all buffers are statically sized at compile time, making it suitable for processors with very limited RAM.

This guide walks you through building a complete DCC command station, then explains every module so you understand exactly what the code does and how to extend it.

1.1 What the Library Does (and Does Not Do)

The library handles all DCC protocol compliance: bit encoding with NMRA-compliant timing (58µs one-bits, 100µs zero-bits), packet construction with XOR error detection, multi-slot scheduling with priority and duplicate combining, all four NMRA service mode methods (direct, paged, register, address), RailCom cutout timing and datagram decoding, and auto-refresh of locomotive speed/function commands.

It does NOT include any hardware-specific code. You supply short driver functions that toggle GPIO pins, start/stop timers, and read current sense, and the library calls them as needed through function pointers in the `dcc_config_t` struct.

1.2 Platform Support

Because the library contains no hardware-specific code, it can be ported to any processor and toolchain. You provide a small set of driver functions for your timer and GPIO hardware, and the library does the rest.

Processor / Board	Example IDE / Toolchain
TI MSPM0G3507	Code Composer Studio / Theia
ESP32	Arduino IDE, PlatformIO
STM32 (F4 and others)	STM32 Cube IDE
Raspberry Pi Pico (RP2040)	Arduino IDE, PlatformIO
Microchip dsPIC33	MPLAB X
Any ARM Cortex-M	GCC + Makefile

This guide focuses on TI MSPM0G3507 as the worked example. All concepts apply equally to any other target.

2. DCC Protocol Concepts

2.1 Signal Encoding

The DCC signal is a square wave where the bit value is encoded in the half-period duration. A one-bit has a nominal half-period of 58µs (NMRA allows 55–61µs). A zero-bit has a minimum half-period of 100µs. The command station alternates the polarity of the track voltage at these intervals.

Bit Value	Half-Period	NMRA Range
One (1)	58 µs nominal	55–61 µs
Zero (0)	100 µs minimum	≥100 µs (command station)

2.2 Packet Format

Every DCC packet follows this structure: Preamble (14+ one-bits) | Start bit (0) | Address byte | Start bit (0) | Instruction byte(s) | Start bit (0) | XOR error check | End bit (1). The XOR byte is the exclusive-or of all preceding data bytes.

2.3 Address Types

Address Type	Range	Bytes
Broadcast	0	1
Short (7-bit)	1–127	1
Long (14-bit)	128–10239	2 (0xC0 prefix)
Idle	255	1
Accessory	1–511 (board)	2
Extended Accessory	1–2048	2

2.4 Instruction Types

Instruction	Mask	Description
Speed (14/28-step)	0x40/0x60	Forward/reverse speed command
Speed (128-step)	0x3F (advanced)	Extended speed with 126 steps
Function Group 1	0x80	FL, F1–F4
Function Group 2a	0xB0	F5–F8
Function Group 2b	0xA0	F9–F12
Feature Expansion	0xC0	F13+, binary state, analog
CV Access (long)	0xE0	Operations mode CV read/write

3. Project File Structure

Understanding the file layout tells you where to look when you need to change something. The files in `dcc_lib/` are library code — do not edit them. The files at the top level and in `application_callbacks/` and `application_drivers/` are yours.

<code>command_station/</code>	<- Your project folder
<code>command_station.c</code>	<- Main entry (setup + loop + ISRs)
<code>dcc_user_config.h</code>	<- REQUIRED: feature flags & buffer sizes
<code>uart_command_parser.c/h</code>	<- Demo UART CLI
<code>application_callbacks/</code>	
<code>callbacks_dcc.c/h</code>	<- Application event hooks
<code>application_drivers/</code>	
<code>ti_driverlib_dcc_driver.c/h</code>	<- Hardware interface (timers, GPIO)
<code>ti_driverlib_uart_driver.c/h</code>	<- UART I/O
<code>dcc_lib/</code>	<- Library core (do not edit)
<code>dcc_config.h/c</code>	- Stack init API
<code>dcc_types.h</code>	- All data types
<code>dcc_defines.h</code>	- NMRA constants
<code>dcc_bit_encoder.h/c</code>	- ISR-level bit serialization
<code>dcc_application_command_station_packet.h/c</code>	- Packet byte construction
<code>dcc_scheduler.h/c</code>	- Multi-slot packet scheduling
<code>dcc_railcom_cutout.h/c</code>	- RailCom cutout state machine
<code>dcc_railcom_decoder.h/c</code>	- RailCom datagram decoder
<code>dcc_service_mode_*.h/c</code>	- Service mode state machines
<code>dcc_application_*.h/c</code>	- Public application API

4. dcc_user_config.h in Depth

This file is the single most important file in your project. The library uses these constants at compile time to determine how much memory to allocate. If a value is wrong, the library will either waste RAM or crash at runtime.

4.1 Role and Service Mode Flags

```
// Uncomment the role(s) your project uses.
#define DCC_COMPILE_COMMAND_STATION

// Service mode methods (each requires DCC_COMPILE_COMMAND_STATION).
#define DCC_COMPILE_SERVICE_MODE_DIRECT    // Direct byte R/W
#define DCC_COMPILE_SERVICE_MODE_PAGED    // Paged addressing
#define DCC_COMPILE_SERVICE_MODE_REGISTER // Register-only (old decoders)
#define DCC_COMPILE_SERVICE_MODE_ADDRESS  // Address-only (CV1 shortcut)
```

Each `DCC_COMPILE_SERVICE_MODE_*` flag requires `DCC_COMPILE_COMMAND_STATION`. The compiler will emit a `#error` if you enable a service mode without the command station role.

4.2 Scheduler Slot Count

`USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT` controls how many packets can be queued simultaneously. Each slot holds one `dcc_scheduler_slot_t` (packet bytes + metadata). The scheduler uses a static array — no dynamic allocation. A typical command station needs 16–24 slots.

4.3 Auto-Refresh Locos

`USER_DEFINED_DCC_MAX_LOCOS` sets how many locomotives the scheduler can auto-refresh. The NMRA standard requires command stations to periodically re-send speed and function commands so decoders do not time out. Each active loco consumes one refresh entry.

4.4 RailCom Buffer

`USER_DEFINED_DCC_RAILCOM_BUFFER_DEPTH` sets the ring buffer depth for received RailCom datagrams. The ISR writes datagrams into this buffer; `DccConfig_run()` drains them to your callback. A depth of 4 is sufficient for most layouts.

4.5 Service Mode Parameters

Define	Default	Description
<code>USER_DEFINED_DCC_SERVICE_MODE_RETRIES</code>	3	Number of retry attempts before reporting failure
<code>USER_DEFINED_DCC_ACK_THRESHOLD_MA</code>	60	Minimum current increase (mA) to detect ACK pulse
<code>USER_DEFINED_DCC_ACK_MIN_DURATION_US</code>	5000	Minimum ACK pulse duration (µs)

USER_DEFINED_DCC_ACK_MAX_DURATION_US	7000	Maximum ACK pulse duration (µs)
--------------------------------------	------	---------------------------------

5. Initialization — command_station.c

The main file wires together the hardware drivers, the library, and your application callbacks. The pattern is always the same four steps: define the config struct, initialize hardware, initialize the library, enter the main loop.

5.1 The `dcc_config_t` Struct

This struct tells the DCC library how to talk to your hardware. All fields are function pointers. The library calls these when it needs to toggle a pin, start a timer, or notify you of an event.

```
static const dcc_config_t dcc_config = {
    // REQUIRED: common platform drivers
    .lock_shared_resources = &TI_DccDriver_lock_shared_resources,
    .unlock_shared_resources = &TI_DccDriver_unlock_shared_resources,
    .get_timestamp_usec = &TI_DccDriver_get_timestamp_usec,

    // Shared 58us timer
    .shared_timer_start = &TI_DccDriver_shared_timer_start,
    .shared_timer_stop = &TI_DccDriver_shared_timer_stop,

    // RailCom cutout one-shot timer
    .railcom_timer_start = &TI_DccDriver_railcom_timer_start,
    .railcom_timer_stop = &TI_DccDriver_railcom_timer_stop,

    // Per-channel hardware
    .main_track = { ... },
    .service_track = { ... },

    // Application callbacks (NULL = no notification)
    .on_packet_sent = &CallbacksDcc_on_packet_sent,
    .on_service_mode_result = &CallbacksDcc_on_service_mode_result,
};
```

5.2 Per-Channel Hardware (`dcc_output_hw_t`)

Each DCC output channel (main track, service track) has its own set of hardware function pointers. The shared timer architecture means `timer_start` and `timer_stop` can be `NULL` — `pin_toggle` is used instead.

Field	Required?	Description
<code>pin_toggle</code>	REQUIRED	Toggle DCC output GPIO pin (ISR context)
<code>track_power_set</code>	REQUIRED	Enable/disable H-bridge
<code>timer_start</code>	NULL w/ shared timer	Start per-channel timer
<code>timer_stop</code>	NULL w/ shared timer	Stop per-channel timer
<code>current_sense_read</code>	Service track only	Read milliamps for ACK detection
<code>railcom</code>	NULL if no RailCom	Pointer to <code>dcc_railcom_hw_t</code>

5.3 RailCom Hardware (`dcc_railcom_hw_t`)

Field	Description
begin_railcom_cutout	Tristate H-bridge output at T_CS (26µs after end bit)
end_railcom_cutout	Restore H-bridge to normal drive at T_CE
uart_rx_enable	Enable 250 kbaud UART for RailCom data
uart_rx_disable	Disable UART after RailCom window
uart_read	Read one byte from RailCom UART (returns true if available)
on_railcom_datagram_result	Decoded datagram callback (from DccConfig_run, not ISR)

5.4 Setup and Main Loop

```
int main(void) {
    SYSCFG_DL_init();           // MCU clocks, GPIO, timers, UART
    TI_DccDriver_initialize();   // Start free-running timestamp
    TI_UartDriver_initialize();  // Enable UART
    DccConfig_initialize(&dcc_config); // Hand config to library
    UartCommandParser_initialize(); // Set up CLI

    while (1) {
        DccConfig_run();        // Library scheduling & callbacks
        TI_UartDriver_echo_process(); // Echo characters
        UartCommandParser_process(); // Execute commands
    }
}
```

DccConfig_run() must be called repeatedly from your main loop. Do not use delay() or blocking calls in the loop — it stalls the scheduler and callback dispatch.

6. ISR Architecture

6.1 The 58µs Shared Timer

A single hardware timer fires every 58µs (DCC_ONE_BIT_HALF_PERIOD_US). Its ISR calls `DccConfig_58us_timer_isr()`, which ticks both the main track and service track bit encoders. Each encoder counts ticks to determine when to toggle its output pin and when to advance to the next bit. Zero-bits take more ticks than one-bits.

```
void DCC_BIT_TIMER_INST_IRQHandler(void) {
    switch (DL_TimerA_getPendingInterrupt(DCC_BIT_TIMER_INST)) {
        case DL_TIMER_IIDX_ZERO:
            TI_DccDriver_timestamp_tick();
            DccConfig_58us_timer_isr();
            break;
        default:
            break;
    }
}
```

6.2 RailCom One-Shot Timer

A second timer operates in one-shot mode, driving a five-state cutout machine. The library restarts it for each phase: DELAY 26µs (tristate H-bridge at $T_{CS}=26\mu s$), SETTling 54µs (enable UART Rx at $T_{TS1}=80\mu s$), CH1 window 97µs ($T_{TC1}=177\mu s$), GAP 16µs ($T_{TS2}=193\mu s$), and CH2 window 261µs (re-enable H-bridge at $T_{CE}=454\mu s$). Total cutout is ~454µs. All five durations are user-configurable in `dcc_config_t`. The ISR calls `DccConfig_railcom_oneshot_timer_isr()`.

6.3 100ms Periodic Tick

A 100ms timer (typically SysTick) calls `DccConfig_100ms_timer_tick()` for timeout checking and periodic housekeeping. This is also a good place for heartbeat LED toggling.

7. Implementing the Drivers

The driver files are the only hardware-specific code in a project. If you are using the MSPM0G3507 example, these are already implemented. This section explains what each function must do when porting to a new platform.

7.1 Platform Driver Functions

Function	Context	Contract
lock_shared_resources	Any	Disable interrupts or acquire mutex. Must be nestable or paired.
unlock_shared_resources	Any	Re-enable interrupts or release mutex.
get_timestamp_usec	Any	Return monotonic microsecond counter. Wrapping at 2^{32} is fine.
shared_timer_start(period)	Main	Start periodic timer with given period. ISR calls <code>DccConfig_58us_timer_isr()</code> .
shared_timer_stop()	Main	Stop the periodic timer.
railcom_timer_start(period)	ISR	Start one-shot timer. ISR calls <code>DccConfig_railcom_oneshot_timer_isr()</code> .
railcom_timer_stop()	ISR	Stop one-shot timer.

7.2 Pin Toggle

Called from ISR context by the bit encoder. Must be a single register write — no disable/re-enable of interrupts. The main track and service track each have their own `pin_toggle` function.

7.3 Track Power

`track_power_set(true)` enables the H-bridge; `track_power_set(false)` disables it. Called from main-loop context.

7.4 Current Sense

`current_sense_read()` returns milliamps (if using ADC) or a threshold value (if using comparator). The service track uses this for ACK detection during service mode programming. The demo returns 100 (HIGH) or 0 (LOW) from a digital pin.

7.5 Timestamp

`get_timestamp_usec()` must return a free-running microsecond counter. It does not need to be absolute — only monotonic. The library handles 32-bit wrap-around (~71 minutes). The demo increments a software counter from the 58μs timer ISR.

8. The Bit Encoder

The bit encoder serializes DCC packets one bit at a time from ISR context. It is single-buffered: it holds one active packet plus a `packet_loaded` flag. The ISR transmits the active packet, and on completion signals the main loop, which loads the next packet via `DccBitEncoder_load_packet()` during the inter-packet window (there is no front/back buffer swap).

With the shared 58µs timer architecture, the encoder counts timer ticks per half-bit. A one-bit = 1 tick (58µs), a zero-bit = 2 ticks (116µs, within NMRA spec). On each tick, the encoder decides whether to toggle the output pin based on the current bit value and tick count.

The encoder walks through the packet structure in order: preamble bits, then for each data byte: start bit (0) + 8 data bits, and finally the end bit (1). When the last bit of a packet is sent, it signals the scheduler to load the next packet.

9. The Scheduler

9.1 Slot Allocation

The scheduler manages a static array of `USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT` slots. Each slot holds one DCC packet (up to 6 bytes), its priority, address, tag, repeat count, and auto-refresh flag. When you call an API like `DccApplicationCommandStationMainTrack_load_speed_128()`, the scheduler finds an available slot (or reuses one with the same address+tag) and fills it.

9.2 Priority System

Priority	Enum Value	Use Case
ESTOP	Highest	Emergency stop commands
SPEED	High	Speed and direction changes
FUNCTION	Medium	Function on/off commands
ACCESSORY	Medium	Turnout and signal commands
CV	Low	Operations mode CV programming
IDLE	Lowest	Idle packets (keep-alive)

9.3 Duplicate Combining

When a new command arrives for the same (address, tag) pair as an existing slot, the scheduler overwrites the old packet with the new one instead of allocating a second slot. Tags distinguish command types: `SPEED`, `FUNC_GROUP_1`, `FUNC_GROUP_2A`, etc. This prevents the scheduler from filling up when rapid-fire commands arrive for the same locomotive.

9.4 Auto-Refresh

The NMRA standard requires command stations to periodically re-send speed and function commands. When a slot is marked for auto-refresh, the scheduler re-queues it after transmission instead of releasing the slot. The scheduler round-robins through active refresh entries between one-shot commands.

10. Service Mode Programming

Service mode operates on a dedicated programming track. The command station sends specific packet sequences, and the decoder responds with a 6ms current pulse (ACK) when a command succeeds.

10.1 Direct Mode

Reads or writes a single CV by number. The command station sends 3 reset packets, then 5 identical direct-mode instruction packets, then monitors current for ACK. This is the most common and reliable service mode method.

10.2 Paged Mode

Divides the CV space into 256-CV pages. A page preset command selects the page, then a register write/verify accesses CVs 1–4 within that page. Useful for older decoders that do not support direct mode.

10.3 Register Mode

Accesses only 8 physical registers (mapped to CV1–CV4, CV7, CV8, CV29). The oldest service mode method. Some very old decoders only support this.

10.4 Address Mode

A shortcut for writing CV1 (primary address). Sends address-only packets that even the simplest decoders understand. Limited to short addresses (1–127).

10.5 ACK Detection

The library samples `current_sense_read()` on the service track during the ACK window. A valid ACK is a current pulse that exceeds `USER_DEFINED_DCC_ACK_THRESHOLD_MA` and whose duration falls within BOTH bounds: at least `USER_DEFINED_DCC_ACK_MIN_DURATION_US` and no more than `USER_DEFINED_DCC_ACK_MAX_DURATION_US`. A pulse shorter than the minimum is ignored; a pulse longer than the maximum is rejected as an over-current condition (not an ACK). A valid pulse reports `SUCCESS`; no qualifying pulse = `NO_ACK` result.

11. RailCom (Brief Overview)

RailCom is a bi-directional communication extension to DCC. After each packet, the command station opens a cutout window (~454µs total) by tristating the H-bridge. During this window, the decoder sends data back on the track as a current-source signal.

The cutout has two channels: Ch1 (short, 2 bytes) for broadcast address data, and Ch2 (long, up to 6 bytes) for addressed data like CV values and locomotive ID. The library handles all cutout timing through the one-shot timer, UART reception, 4/8 decoding, and datagram assembly. All five cutout-phase durations are user-configurable in `dcc_config_t`. Your callback receives fully decoded datagrams.

Cutout State	Duration / boundary	Description
DELAY	26 µs → T_CS = 26 µs	Tristate the H-bridge after the end bit
SETTLING	54 µs → T_TS1 = 80 µs	Line settles; enable UART Rx
CH1	97 µs → T_TC1 = 177 µs	Decoder broadcasts address (ID), up to 2 bytes
GAP	16 µs → T_TS2 = 193 µs	Idle between Ch1 and Ch2; re-arm UART Rx
CH2	261 µs → T_CE = 454 µs	Addressed data (CV value, status), up to 6 bytes; H-bridge re-enabled

12. Callbacks — Where Your Application Lives

Callbacks are function pointers in the `dcc_config_t` struct that the library calls when specific events occur. All callbacks fire from `DccConfig_run()` context (main loop), NOT from ISR context, so they are safe to use UART or other slow I/O.

`on_packet_sent`

Called after every DCC packet is fully transmitted on the track. Useful for debugging (scope trigger, UART log) or statistics.

```
void CallbacksDcc_on_packet_sent(const dcc_packet_t *packet) {
    (void)packet;
    DL_GPIO_togglePins(GPIO_DEBUG_PORT, GPIO_DEBUG_DEBUG_PIN_PIN);
}
```

`on_service_mode_result`

Called when a service mode operation completes. The result enum tells you whether it succeeded, failed with no ACK, or encountered an error.

Result	Meaning
DCC_SERVICE_MODE_SUCCESS	CV operation completed with ACK from decoder
DCC_SERVICE_MODE_NO_ACK	No current pulse detected — no decoder on track?
DCC_SERVICE_MODE_VERIFY_FAIL	Verify command: value does not match
DCC_SERVICE_MODE_BUSY	Another operation is already in progress
DCC_SERVICE_MODE_ERROR	Internal error
DCC_SERVICE_MODE_NOT_IN_SERVICE_MODE	Must call SVC ENTER first

13. Application API Reference

These are the functions you call from your application to command locomotives, accessories, and service mode operations. All are defined in the `dcc_application_command_station_*.h` headers.

Main Track Operations

Function	Description
<code>_load_speed_14(addr, speed, dir, fl)</code>	14-step speed command
<code>_load_speed_28(addr, speed, dir)</code>	28-step speed command
<code>_load_speed_128(addr, speed, dir)</code>	128-step speed command
<code>_load_emergency_stop(addr)</code>	E-stop specific loco
<code>_load_emergency_stop_all()</code>	Broadcast e-stop
<code>_load_function_group_1(addr, fl, f1-f4)</code>	FL and F1–F4
<code>_load_function_group_2a(addr, f5-f8)</code>	F5–F8
<code>_load_function_group_2b(addr, f9-f12)</code>	F9–F12
<code>_load_function_f13_f20(addr, bits)</code>	F13–F20
<code>_load_function_f21_f28(addr, bits)</code>	F21–F28
<code>_load_accessory_basic(board, pair, activate)</code>	Turnout throw/close
<code>_load_accessory_extended(addr, aspect)</code>	Signal aspect
<code>_load_cv_write(addr, cv, value)</code>	Ops mode CV write (POM)
<code>_load_cv_verify(addr, cv, value)</code>	Ops mode CV verify
<code>_power_on()</code> / <code>_power_off()</code>	Track power control

Service Track Operations

Function	Description
<code>_enter_service_mode()</code>	Switch to programming mode
<code>_exit_service_mode()</code>	Return to normal operations
<code>_direct_write(cv, value)</code>	Write CV in direct mode
<code>_direct_verify(cv, value)</code>	Verify CV in direct mode
<code>_direct_bit_write(cv, bit, value)</code>	Write single CV bit
<code>_direct_bit_verify(cv, bit, value)</code>	Verify single CV bit
<code>_paged_write(cv, value)</code>	Write CV in paged mode

<code>_paged_verify(cv, value)</code>	Verify CV in paged mode
<code>_register_write(reg, value)</code>	Write register
<code>_register_verify(reg, value)</code>	Verify register
<code>_address_write(addr)</code>	Write short address (CV1)
<code>_address_verify(addr)</code>	Verify short address

14. Porting to a New MCU

Porting to a new processor means writing the driver functions that match the function pointer signatures in `dcc_config_t`. The library never touches hardware directly.

1. Copy the `command_station` example project to a new folder.
2. Replace `ti_driverlib_dcc_driver.c/h` with your MCU's timer and GPIO API.
3. Replace `ti_driverlib_uart_driver.c/h` with your MCU's UART API (optional, for CLI).
4. Update `dcc_user_config.h` buffer sizes if your MCU has limited RAM.
5. Wire your ISRs to call `DccConfig_58us_timer_isr()`, `DccConfig_railcom_oneshot_timer_isr()`, and `DccConfig_100ms_timer_tick()`.
6. Build and test.

The minimum hardware requirements are: one periodic timer (~58µs capable), one GPIO output per track channel (toggled from ISR), and a microsecond timestamp source. Optional: one-shot timer for RailCom, ADC/comparator for service mode ACK, UART for RailCom detector.

15. Unit Testing

The library includes comprehensive unit tests using Google Test (C++). Tests are located alongside the source files as *_Test.cxx files in src/dcc/.

Running the Tests

```
cd test
mkdir build &&& cd build
cmake ..
make
ctest --output-on-failure
```

Test Suites

Test File	What It Tests
dcc_bit_encoder_Test.cxx	Bit timing, preamble generation, packet framing
dcc_scheduler_Test.cxx	Slot allocation, priority, duplicate combining, auto-refresh
dcc_packet_encoder_Test.cxx	Packet byte construction for all instruction types
dcc_config_Test.cxx	Initialization, lifecycle, ISR dispatch
dcc_service_mode_direct_Test.cxx	Direct mode CV read/write state machine
dcc_service_mode_paged_Test.cxx	Paged mode state machine
dcc_railcom_cutout_Test.cxx	Cutout timing state machine
dcc_railcom_decoder_Test.cxx	4/8 datagram decoding
dcc_bit_decoder_Test.cxx	Edge timing classification (one/zero)
dcc_packet_decoder_Test.cxx	Packet assembly, XOR validation, address matching
dcc_cv_storage_Test.cxx	CV read/write with decoder lock

Coverage

```
# Generate HTML coverage report
cmake -DCMAKE_BUILD_TYPE=Coverage ..
make
ctest
gcovr --html-details coverage.html -r ../../src/dcc/
```

16. Troubleshooting

Symptom	Likely Cause and Fix
No DCC signal on scope	Track power is OFF. Type POWER ON. Check pin_toggle wiring.
Signal timing wrong	Check shared timer period is 58μs. Verify MCU clock configuration.
Decoder does not respond	Check H-bridge wiring. Verify decoder address matches SPEED command.
SVC RESULT: NO ACK	No decoder on programming track. Check current_sense_read wiring. Verify ACK threshold (60 mA default).
SVC RESULT: BUSY	A service mode operation is already running. Wait for it to complete.
Scheduler full	Increase USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT in dcc_user_config.h.
Undefined reference to USER_DEFINED_*	dcc_user_config.h is not on the include path. Add -I flag pointing to its directory.
Compiler #error: requires DCC_COMPILE_COMMAND_STATION	You enabled a service mode flag without the command station role. Add #define DCC_COMPILE_COMMAND_STATION.
RailCom datagrams not received	Check 250 kbaud UART wiring and dcc_railcom_hw_t struct. Verify railcom_timer_start/stop are wired.

Online Resources

Library source: github.com/jimkueneman/OpenDccCLib

NMRA DCC standards: www.nmra.org/dcc